

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ
к лабораторной работе № 4
на тему «Асимметричная криптография. Алгоритм Мак-Элиса»

Выполнил

Е. С. Кахновский

Проверил

А. В. Герчик

Минск 2024

СОДЕРЖАНИЕ

1 Цель лабораторной работы	3
2 Теоретические сведения	4
3 Результат выполнения	4
Вывод.....	6
Список использованных источников	7
Приложение А (обязательное) Листинг исходного кода	8

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Целью выполнения данной лабораторной работы является реализация программного средства шифрования и дешифрования текстовых файлов при помощи Алгоритм Мак-Элиса.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

McEliece — криптосистема с открытыми ключами на основе теории алгебраического кодирования, разработанная в 1978 году Робертом Мак-Элисом[англ.][1]. Это была первая схема, использующая рандомизацию в процессе шифрования. Алгоритм не получил широко признания в криптографии, но в то же время является кандидатом для постквантовой криптографии, так как устойчив к атаке с использованием Алгоритма Шора.

Алгоритм основан на сложности декодирования полных линейных кодов (общая задача декодирования является NP-сложной).

Для описания закрытого ключа выбран код исправляющий ошибки, для которого известен эффективный алгоритм декодирования и который может исправить ошибок.

Алгоритм использует двоичные коды Гоппа, которые легко декодируются благодаря алгоритму Петерсона. Открытый ключ получается при помощи маскировки выбранного кода как полного линейного. Для этого порождающая матрица

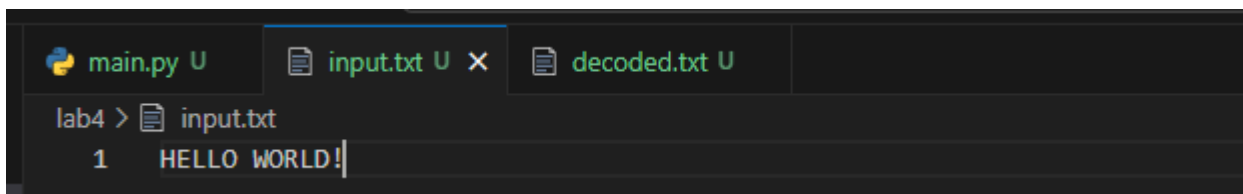
Существует несколько вариантов криптосистемы, использующих различные типы кодов. Большинство из них оказываются менее защищёнными. Отдельного рассмотрения заслуживает вопрос выбора параметров криптосистемы.

До сих пор McEliece с кодами Гоппы не поддаётся криптоанализ. Наиболее известные атаки используют алгоритм декодирования множества данных. Последние работы описывают как атаки на систему, так и её защиту. В других работах показано, что для квантовых вычислений размер ключа должен быть увеличен на четыре порядка из-за усовершенствования декодирования множества данных.

Криптосистема имеет несколько преимуществ, например, перед RSA. Шифрование и дешифрование проходит быстрее и с ростом длины ключа степень защиты данных растёт гораздо быстрее. Долгое время считалось, что McEliece не может быть использована для ЭЦП. Однако оказалось возможным построить схему для ЭЦП на основе криптосистемы Нидеррайтера (модификация McEliece). Поскольку в зависимости от используемых кодов, эти две криптосистемы выражают одну и ту же кодовую задачу, на сегодняшний день, можно утверждать, что McEliece применим в задачах аутентификации.

3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

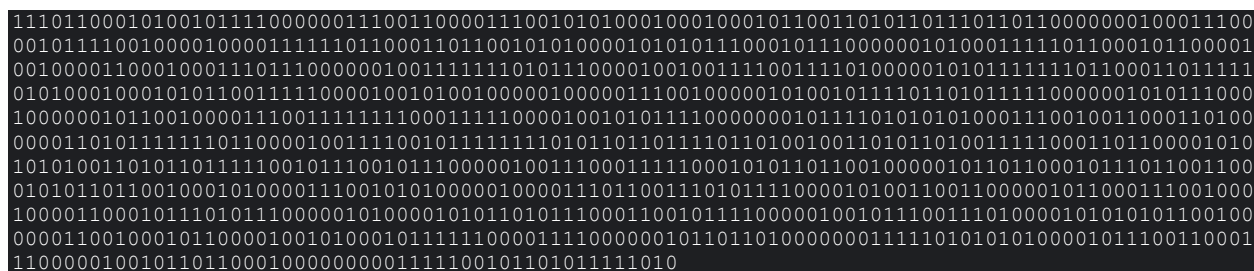
В ходе выполнения лабораторной было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи Алгоритм Мак-Элиса.



The screenshot shows a code editor with three tabs: 'main.py U', 'input.txt U', and 'decoded.txt U'. The 'input.txt U' tab is active, showing the text 'lab4 > input.txt' followed by a line number '1' and the text 'HELLO WORLD!'.

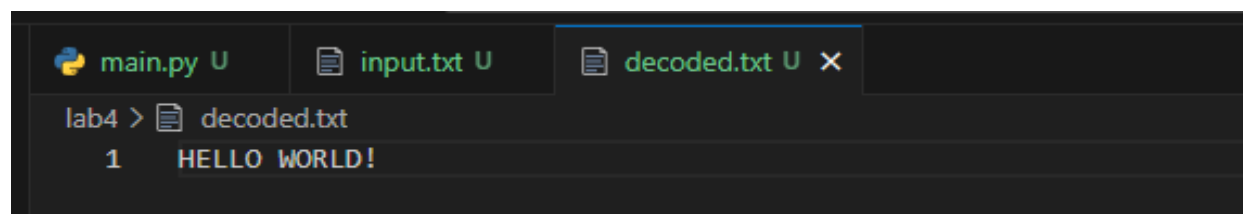
Рисунок 1 – Запись исходного текста в файл input.txt

В результате выполнения зашифрованный текст сохраняется в файл encrypted.txt.



The screenshot shows a code editor with a single tab containing a long string of binary code (0s and 1s) representing the encrypted text.

Рисунок 2 –Зашифрованный текст в encrypted.txt



The screenshot shows a code editor with three tabs: 'main.py U', 'input.txt U', and 'decoded.txt U'. The 'decoded.txt U' tab is active, showing the text 'lab4 > decoded.txt' followed by a line number '1' and the text 'HELLO WORLD!'.

Рисунок 2 – Запись расшифрованного текста в файл decrypted.tx

ВЫВОД

В ходе выполнения данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов с использованием Криптосистемы Мак-Элиса. Этот процесс включал в себя несколько важных шагов, включая генерацию ключей, шифрование и последующую дешифрацию данных.

В итоге, выполнение данной лабораторной работы позволило нам приобрести практические навыки в области асимметричной криптографии и ознакомиться с принципами работы Криптосистемы Мак-Элиса.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] McEliece [Электронный ресурс]. – Режим доступа:
<https://ru.wikipedia.org/wiki/McEliece> – Дата доступа: 03.11.2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

```
import random
import numpy as np

# Проверочная матрица
H = np.array([[1, 0, 1, 0, 1, 0, 1],
              [0, 1, 1, 0, 0, 1, 1],
              [0, 0, 0, 1, 1, 1, 1]])

# Матрица генерации
G = np.array([[1, 1, 0, 1],
              [1, 0, 1, 1],
              [1, 0, 0, 0],
              [0, 1, 1, 1],
              [0, 1, 0, 0],
              [0, 0, 1, 0],
              [0, 0, 0, 1]])

"""Эта матрица используется для декодирования закодированных сообщений.
Она позволяет извлечь исходные 4 бита из 7-битного закодированного
сообщения."""
R = np.array([[0, 0, 1, 0, 0, 0, 0],
              [0, 0, 0, 0, 1, 0, 0],
              [0, 0, 0, 0, 0, 1, 0],
              [0, 0, 0, 0, 0, 0, 1]])

def random_binary_non_singular_matrix(n):
    a = np.random.randint(0, 2, size=(n, n))
    while np.linalg.det(a) == 0:
        a = np.random.randint(0, 2, size=(n, n))
    return a

S = random_binary_non_singular_matrix(4)
S_inv = np.linalg.inv(S).astype(int)

def generate_permutation_matrix(n):
    i = np.eye(n)
    p = np.random.permutation(i)
    return p.astype(int)

P = generate_permutation_matrix(7)
P_inv = np.linalg.inv(P).astype(int)

G_hat = np.transpose(np.mod((S.dot(np.transpose(G))).dot(P), 2))

# Определяет позицию ошибки в закодированных данных.
def detect_error(err_enc_bits):
    err_idx_vec = np.mod(H.dot(err_enc_bits), 2)
    err_idx_vec = err_idx_vec[::-1]
    err_idx = int(''.join(str(bit) for bit in err_idx_vec), 2)
    return err_idx - 1

def hamming7_4_encode(p_str):
```

```

p = np.array([int(x) for x in p_str])

prod = np.mod(G_hat.dot(p), 2)
return prod

def hamming7_4_decode(c):
    prod = np.mod(R.dot(c), 2)
    return prod

def flip_bit(bits, n):
    bits[n] = (bits[n] + 1) % 2

def add_single_bit_error(enc_bits):
    error = [0] * 7
    idx = random.randint(0, 6)
    error[idx] = 1
    return np.mod(enc_bits + error, 2)

dec_msg = []
for enc_bits in enc_msg:
    enc_bits = np.array([int(x) for x in enc_bits])
    # Вычисляем c_hat = c * P_inv
    c_hat = np.mod(enc_bits.dot(P_inv), 2)
    # находим бит ошибки
    err_idx = detect_error(c_hat)
    # переворачиваем бит ошибки
    flip_bit(c_hat, err_idx)
    # находим m_hat
    m_hat = hamming7_4_decode(c_hat)
    # находим m = m_hat * S_inv
    m_out = np.mod(m_hat.dot(S_inv), 2)

    str_dec = ''.join(str(x) for x in m_out)
    dec_msg.append(str_dec)

dec_msg_str = ''.join(dec_msg)
txt = bits_to_str(dec_msg_str)
print(txt)
with open("decoded.txt", "w", encoding="utf-8") as f:
    f.write(txt)
except Exception as e:
    print(f"Error {e}")

elif choice == "4":
    print("Exit from program")
    break

else:
    print("Restart")

if __name__ == "__main__":
    main()

```